

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 05 -

CONTAINERS PARA ML

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	12
REFERÊNCIAS.....	13

EMSE

O QUE VEM POR AÍ?

Nesta aula, vamos aprender a importância da utilização de containers no ciclo de desenvolvimento de software, utilizando como exemplo uma aplicação similar a um model registry.

EXEMPLO

HANDS ON

Nesta aula prática, vamos criar uma aplicação de forma containerizada, que vai utilizar o pacote desenvolvido na aula anterior. O conteúdo desta aula pode ser encontrado na branch **containers** do repositório https://github.com/rocaabrera/curso_mlops/tree/containers.

EMSE

SAIBA MAIS

Sabemos que não basta desenvolver o software; precisamos colocá-lo em funcionamento para os clientes, e esse processo é chamado de “deploy” em produção. Logo, é interessante desenvolver da maneira mais reprodutível possível, pois você precisará reproduzir a execução da sua aplicação em algum servidor.

A utilização de ambientes virtuais é o começo, o ambiente virtual permite fixar os pacotes utilizados, mas às vezes a aplicação **depende** de alguma peculiaridade que não está relacionada a linguagem de programação que estamos utilizando, por exemplo uma variável de ambiente. Nesta aula, vamos aprender sobre containers, uma tecnologia que ajuda a tornar o desenvolvimento de software mais reprodutível, escalável e maleável para mudanças de hardware, como por exemplo, trocar a aplicação de uma nuvem para outra.

Atualmente, precisamos realizar o deploy de diferentes tipos de software, como bancos de dados relacionais, serviços de fila, endpoints, entre outros. No entanto, há diversas opções de hardware disponíveis, desde laptops a servidores robustos. Os containers oferecem uma padronização que permite a execução desses serviços de maneira intercambiável em diferentes hardwares. De forma análoga, existe a padronização de transporte de produtos via containers, conforme mostra a Figura 1:

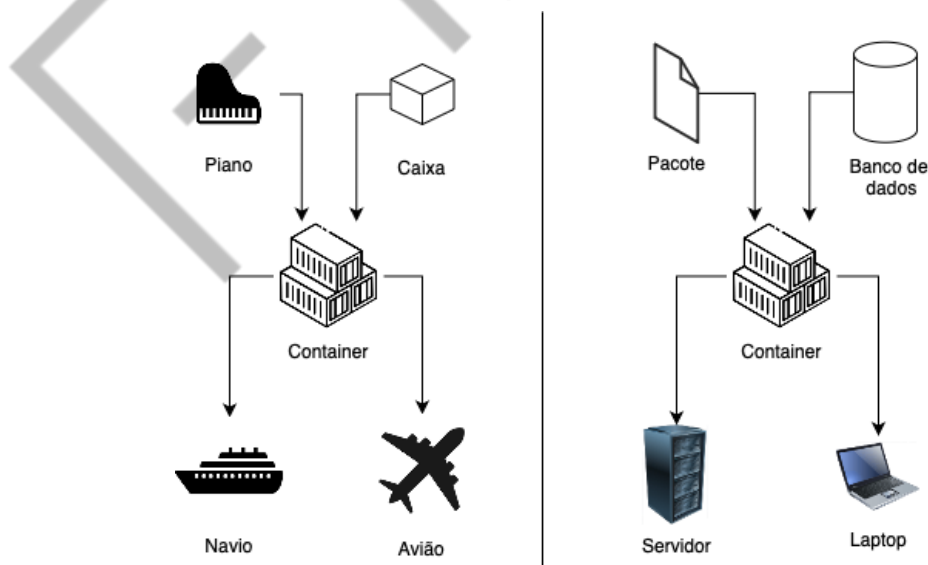


Figura 1 - Analogia sobre a utilização de containers em software e produtos físicos
Fonte: Elaborado pelo autor (2024)

Esses produtos (softwares) são colocados em containers (padronização) que são transportados por navios de diferentes marcas e modelos (diferentes hardwares). Portanto, qualquer meio capaz de transportar containers é capaz de movimentar esses produtos (realizar o deploy).

Basicamente, se você construir um container que funciona na sua máquina local, seu código vai funcionar em qualquer lugar (existem algumas exceções).

A ferramenta mais famosa para construir esses containers é o Docker. Quando utilizamos Docker existem dois principais conceitos para aprendermos:

Imagens: conceitualmente similares às Classes.

Containers: conceitualmente similares a objetos instanciados por uma classe.

Assim como os pacotes Python ficam em um índice, as imagens Docker ficam salvas em um registro de containers (container registry), sendo o [Docker Hub](#) o mais conhecido. No entanto, existem também registros de containers proprietários, como o ECR (Elastic Container Registry) da AWS (Amazon Web Services).

Primeiro, instale o docker no seu sistema operacional (Se você estiver utilizando o CodeSpace do Github não precisa, ele já vem instalado). Quando você digitar no terminal o comando **docker version**, deve aparecer algo semelhante a seguinte mensagem:

Docker version 24.0.2, build cb74dfc

A maioria dos comandos do docker seguirão o seguinte padrão:

docker <command> <sub-command> [options]

Porém, alguns comandos podem seguir o padrão antigo:

docker <sub-command> [options]

De qualquer forma, a maioria dos comandos são retrocompatíveis com versões anteriores, logo é possível encontrar na internet alguns comandos diferentes dos apresentados nesta aula e que fazem a mesma coisa.

As principais categorias de comando são:

- Image.

- Container.
- Volume.
- Network.

Nesta aula **não** vamos explorar todas essas categorias. Para cada categoria existe um conjunto de subcomandos, como exemplo:

- docker image ls.
- docker container ls.
- docker network ls.
- docker volume ls.

Veja na videoaula as dicas de como lembrar os comandos.

Vamos, agora, construir nossa primeira imagem. Elas funcionam como uma receita, isto é, uma sequência de comandos em uma ordem específica. Ela normalmente começa com o comando **FROM**, como exemplo:

FROM python:3.9

Coloque esse conteúdo em um arquivo chamado MyFirstDockerfile e execute o seguinte comando:

docker image build -f MyFirstDockerfile .

Você pode ler esse comando da seguinte maneira: Vamos buildar a imagem que está no diretório local, com nome de arquivo igual a MyFirstDockerfile. Entretanto, caso a sua imagem (arquivo com o comando FROM) esteja em outro diretório, execute este comando:

docker image build -f ./dockerfolder/MyFirstDockerfile dockerfolder

Você pode ler esse comando da seguinte maneira: vamos “buildar” a imagem que está no diretório dockerfolder, com nome de arquivo igual a “MyFirstDockerfile”. Aparecerá algo semelhante em seu terminal, conforme exemplifica a Figura 2:

```
[+] Building 11.3s (4/5)
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 52B
=> [internal] load metadata for docker.io/library/python:3.9
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/1] FROM docker.io/library/python:3.9@sha256:d05169bc6332f6ec7686be45843c8fa66a1e2e76b28d0d3b5fa6ac7c9ff2cb6d
=> => resolve docker.io/library/python:3.9@sha256:d05169bc6332f6ec7686be45843c8fa66a1e2e76b28d0d3b5fa6ac7c9ff2cb6d
=> sha256:8ec2432c135289b9085d62bab10b2be4db9a518aa092024c970c5385f009e8 2.52kB / 2.52kB
=> sha256:deb23f81edc83f66a2112e31c9804432804d314435e498f267d05908ed0634 7.26kB / 7.26kB
=> sha256:903681d8777d728dc56866a07a2774c3fd5b6f5fd734b24c9d0ecd9a13c9f636 49.55MB / 49.55MB
=> sha256:d05169bc6332f6ec7686be45843c8fa66a1e2e76b28d0d3b5fa6ac7c9ff2cb6d 10.35kB / 10.35kB
=> sha256:3cbb8e6a28c2f6b3c3e0e8c6dcfba369e1ea656cf8daf69be789e0fe2105982b 24.05MB / 24.05MB
=> sha256:6ed93aa58a52c9abc1ee472f1ac74b73d3adccc2c30744498fd5f14f3f5d22c 64.14MB / 64.14MB
=> extracting sha256:903681d8777d728dc56866a07a2774c3fd5b6f5fd734b24c9d0ecd9a13c9f636 2.8s
=> sha256:74e6988984f1813573a79095b3bfab276dfd14f223bcb7b46d2da94226ec27746 6.16MB / 6.16MB
=> sha256:787c78da43830be6d988d34c7ee091f98d828516ce5478ca10a4933d655191bf 211.24MB / 211.24MB
=> sha256:8f13d318446733c0edcf3d47378f7c3eafe757e9a4f0cfe53c436f78eb58 15.82MB / 15.82MB
=> sha256:a28b5664f47e43d58d2a7bf51c88890cd051d7b506b4d4876fce0b496b5889 230B / 230B
=> sha256:777e0be36342451bc6d202f5598edc4f1502adafea353980c1ea3fa9ac301a50 2.70MB / 2.70MB
=> extracting sha256:3cbb8e6a28c2f6b3c3e0e8c6dcfba369e1ea656cf8daf69be789e0fe2105982b 0.6s
=> extracting sha256:6ed93aa58a52c9abc1ee472f1ac74b73d3adccc2c30744498fd5f14f3f5d22c 2.6s
=> extracting sha256:787c78da43830be6d988d34c7ee091f98d828516ce5478ca10a4933d655191bf 0.1s
```

Figura 2 - Exemplo de build de uma imagem
Fonte: Elaborado pelo autor (2024)

Agora, vamos ver essa imagem com o comando:

```
docker image ls
```

Aparecerá algo em seu terminal semelhante a Figura 3:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
<none>	<none>	deb23f81edc8	2 weeks ago	996MB

Figura 3 - Exemplo de output do comando docker image ls
Fonte: Elaborado pelo autor (2024)

Perceba que ela tem uma propriedade chamada **IMAGE ID**, e toda vez que for necessário referenciar a imagem, precisaremos saber o ID. Entretanto, você pode dar um apelido para essa imagem repetindo o comando de construção de imagem utilizando o parâmetro **-t**, como vemos a seguir:

```
docker image build -f MyFirstDockerfile -t my_image .
```

Aparecerá algo em seu terminal semelhante a Figura 4:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
my_image	latest	deb23f81edc8	2 weeks ago	996MB

Figura 4 - Exemplo de imagem com tag
Fonte: Elaborado pelo autor (2024)

Usualmente utilizamos o nome de arquivo default **Dockerfile** e executamos o comando no mesmo diretório do arquivo. Assim sendo, o comando fica:

```
docker image build -t my_image .
```

Com a imagem criada, podemos desenvolver quantos containers quisermos com base nela (a analogia é que o container é uma instância e a imagem é uma classe) utilizando o seguinte comando:

```
docker container run my_image
```

Perceba que aparentemente nada aconteceu, mas se você digitar o comando:

```
docker container ps -a
```

Vai aparecer em seu terminal algo semelhante a Figura 5 (assista a videoaula para entender todos os campos).

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
2a7f1c8e7727	my_image	"python3"	59 seconds ago	Exited (0) 57 seconds ago		laughing_mclean

Figura 5 - Exemplo de container parado na máquina
Fonte: Elaborado pelo autor (2024)

O container foi criado e terminou sua execução, mas ele não foi deletado. Esse é o comportamento padrão, pois você poderia executar esse container caso necessário. Entretanto, para não criarmos um monte de containers que ficarão ocupando espaço na máquina de forma desnecessária, execute o mesmo comando anterior adicionando o parâmetro **--rm**:

```
docker container run --rm my_image
```

Este parâmetro indica que o container deve ser deletado após terminar sua execução.

Agora vamos começar a construir nossa aplicação. No repositório da aula você vai encontrar o folder **app**, o qual contém todos os arquivos que vamos precisar (assista a videoaula para entender mais sobre a construção desse arquivo). Vamos iterar no arquivo Dockerfile configurado conforme a Figura 6.

```
app > Dockerfile > ...
1 FROM python:3.11
2
3 WORKDIR /app
4
5 COPY ./src /app/src/
6 COPY ./requirements /app/requirements/
7
8 RUN pip install --no-cache-dir -U -r requirements/private.txt \
9     && pip install --no-cache-dir -U -r requirements/public.txt
10
11 CMD ["fastapi", "run", "src/main.py", "--port", "8000"]
```

Figura 6 - Arquivo Dockerfile do código da aula
Fonte: Elaborado pelo autor (2024)

Perceba que o folder **app** contém um folder **src** com o código-fonte da nossa aplicação. Já o outro folder **requirements** contém as dependências públicas (pacotes open-source) e dependências privadas (pacote que foi desenvolvido na aula de empacotamento de código). Nesse arquivo Dockerfile estamos utilizando alguns comandos novos:

- **WORKDIR**: funciona de forma semelhante ao comando **cd**, mas cria o folder caso ele não exista.
- **RUN**: executar os mesmos comandos utilizados no terminal.
- **COPY**: traz os arquivos da sua máquina para o container.

Vamos buildar essa imagem com o apelido “application” utilizando o comando:

```
docker build -t application
```

Depois, vamos criar um container a partir dessa imagem utilizando o comando:

```
docker container run -p 8000:8000 --rm application
```

Para interagir com a aplicação, você pode utilizar o notebook **client.ipynb** fornecido no repositório da aula.

Perceba, que no caso dessa aplicação, nós não colocamos o modelo na imagem, isto é, o modelo não foi um dos arquivos copiados para o container. A depender da situação, pode ser interessante, mas existem prós e contras em todas as abordagens. Por exemplo, caso os pesos do modelo estejam na imagem, sua

aplicação é auto-contida, mas ao mesmo tempo a imagem pode ficar maior (ou como se diz popularmente, mais pesada), tornando o processo de escalonamento mais lento. Se você decidir por colocar o modelo na imagem, provavelmente é interessante você colocar o comando de **COPY** no começo do arquivo. Dessa forma, durante o desenvolvimento da aplicação, o docker reaproveita essa Layer tornando o processo de desenvolvimento mais amigável.

Além disso, se tratando de containers para machine learning é interessante saber que hoje podemos nos comunicar de forma muito simples com as GPUs, basta adicionar o parâmetro **--gpus all**, conforme especificado na [documentação](#).

Por último, se você estiver trabalhando com algum provedor de cloud, será possível encontrar imagens bases específicas para cada caso de uso. Por exemplo, a [Amazon](#) tem imagens específicas para uso em seu orquestrador de container o “ECS” (do inglês, Elastic Container Service).

O QUE VOCÊ VIU NESTA AULA?

Nessa aula, vimos a utilização da tecnologia Docker na construção de uma aplicação de machine learning, a fim de tornar a execução da aplicação mais reprodutível e agnóstica a hardware.

EMEND

REFERÊNCIAS

DOCKER. **Packaging your software.** 2024. Disponível em: <https://docs.docker.com/build/building/packaging/>. Acesso em: 10 set. 2024.

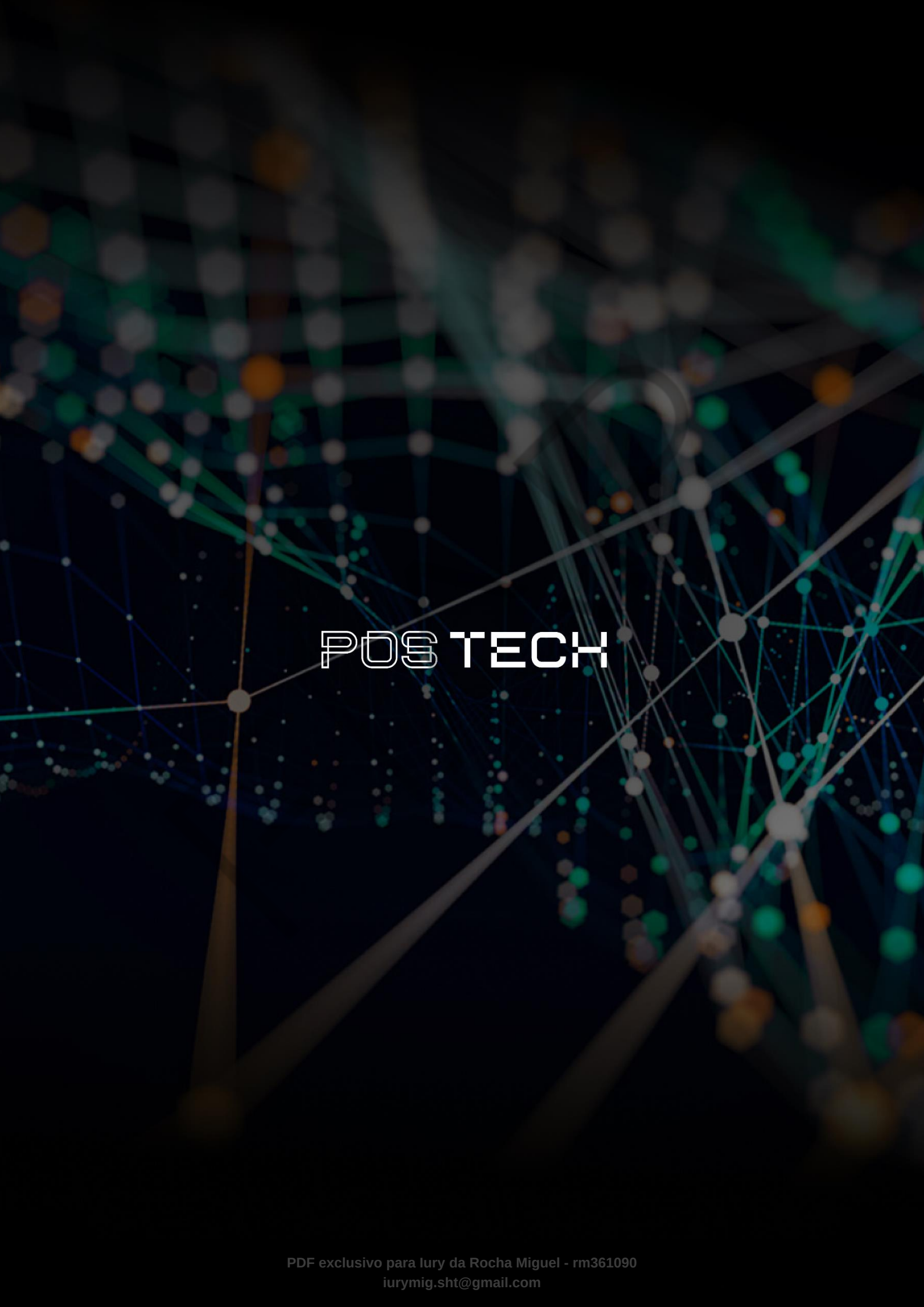
DOCKER. **Running containers.** 2024. Disponível em: <https://docs.docker.com/engine/containers/run/>. Acesso em: 10 set. 2024.

PETAZONNI, J. **Introduction to containers.** 2024. Disponível em: <https://container.training/intro-selfpaced.yml.html#156>. Acesso em: 10 set. 2024.

PALAVRAS-CHAVE

Palavras-chave: Containers. Reprodutibilidade. Reúso. Virtualização

EMENDAS



POSTECH